

Revisão Total da Aplicação

Manutenção — Estádio Municipal de Leiria · PWA offline-first (Vite + JavaScript puro + Dexie/IndexedDB + PostgreSQL)
Data: 17 de agosto de 2026 · Commit: `b4d6e1f` · 15 802 linhas em 41 ficheiros de código

1. Resumo em Uma Página

A app está **funcional e bem pensada nas bases**. A camada de dados (Dexie + fila de sincronização) está sólida, os 120 testes unitários passam todos, e a sincronização foi desenhada com a regra correta: *nunca perder uma avaria*. Os comentários em português no código mostram decisões de desenho conscientes e justificadas — isso é raro e é bom.

Os problemas concentram-se em **três áreas**:

- **Um bug que apaga trabalho do técnico** — o botão "Gravar Intervenção" na folha rápida falha em silêncio quando o técnico escreve um local à mão em vez de o escolher da lista.
- **Duas telas fazem a mesma coisa com os mesmos IDs de HTML** — a captura rápida existe duas vezes (no ecrã Hoje e numa janela flutuante), com IDs duplicados. Isto é a maior fonte de código espagete da app.
- **O desenho visual vive em dois sítios ao mesmo tempo** — há um sistema de tokens CSS excelente em `theme.css`, mas 220 estilos escritos à mão dentro do JavaScript ignoram-no. O tema escuro está meio quebrado por causa disso.

120/120

Testes unitários a passar

220

Estilos `style=""` no JS

16×

Função `esc()` duplicada

78

`!important` no CSS

Área	Nota	Veredicto curto
Camada de dados / repositórios	8,5 / 10	Bem feita. Transações atómicas, fila de sincronização segura, datas em hora local corretas.
Motor de sincronização	8 / 10	Excelente tolerância a falhas. Falta resolver conflitos e apagar itens presos na fila.
Segurança	6 / 10	Sem autenticação nenhuma. API aberta com CORS <code>*</code> . Aceitável em rede fechada, não em internet.
Arquitetura do código	5 / 10	<code>main.js</code> com 1 176 linhas faz de tudo. HTML gerado por texto em todo o lado.
Desenho visual (UI)	5 / 10	Tokens ótimos, mas contornados. Duas fontes de verdade para as mesmas cores.
Utilização (UX) / nº de toques	6 / 10	Gravar é rápido (3 toques). Encontrar coisas antigas é lento. Falta procurar por foto/QR.
Acessibilidade	4 / 10	Alvos de toque bons. Sem gestão de foco nos modais, sem tecla Escape, sem leitor de ecrã.

Código morto ("ghost code")	3 / 10	3 ficheiros e 5 caminhos de navegação nunca usados. Um ficheiro com nome absurdo.
-----------------------------	---------------	---

Se só puder fazer três coisas esta semana

1. Corrigir o bug do local escrito à mão (30 minutos, evita perda de trabalho).
2. Apagar a captura rápida duplicada do ecrã Hoje (2 horas, remove 150 linhas e os IDs duplicados).
3. Apagar os 3 ficheiros mortos e os caminhos de navegação inúteis (20 minutos).

2. Erros a Corrigir — por Ordem de Urgência

CRÍTICO 1 Gravar avaria falha em silêncio

Onde: `src/ui/quickCapture.js:211` e `src/db/reportsRepo.js:108`

O que acontece: Se o técnico escreve o nome do local com o teclado em vez de o escolher da lista, o código põe o identificador do local a `null`. O repositório rejeita isso e lança um erro. O técnico vê "Erro ao guardar intervenção" e **perde o que escreveu**.

Porque é grave: Escrever à mão é o caminho natural quando o local não está na lista. E o ecrã Hoje já tem a correção certa (usa `LOC_UNKNOWN`), mas a folha rápida não. Duas cópias do mesmo código, uma corrigida e outra não — isto é exatamente o custo da duplicação.

Correção

Em `quickCapture.js`, antes de criar: `locationId: this.selectedLocId || 'LOC_UNKNOWN'`.
E fazer com que o botão não perca o texto se o gravar falhar — manter a janela aberta.

CRÍTICO 2 IDs de HTML duplicados no mesmo ecrã

Onde: `src/ui/homeView.js` e `src/ui/quickCapture.js`

O que acontece: Os identificadores `qc-description`, `qc-loc-search`, `qc-loc-dropdown` e `btn-save-capture` existem nos **dois** ficheiros. Quando a folha rápida abre por cima do ecrã Hoje, existem dois elementos com o mesmo identificador na página. Qualquer código que use `document.getElementById` apanha o errado.

Sintoma real: A escrita por voz pode escrever no campo escondido por baixo em vez do campo visível.

Correção

Apagar o bloco de captura rápida do `homeView.js` (linhas 62–91 e os eventos ligados). O ecrã Hoje passa a ter um botão grande que abre a folha rápida. Uma só cópia, um só sítio para corrigir.

CRÍTICO 3 Fila de sincronização pode crescer para sempre

Onde: `src/services/syncEngine.js:218`

O que acontece: Quando o envio falha, a fila fica intacta — isso está *certo*. Mas se uma mutação for sempre rejeitada pelo servidor (por exemplo, um campo demasiado longo para a coluna da base de dados), fica na fila para sempre. O campo `retryCount` existe no esquema mas **nunca é incrementado nem lido**.

Cada avaria com fotos leva um `dataURL` em base64. Cinco fotos são cerca de 1 MB no pedido. Uma fila com 50 itens presos são 50 MB por tentativa, a cada 30 segundos, no telemóvel do técnico.

Correção

Contar as tentativas por item. Ao chegar a 5, mover para uma tabela `sync_dead_letter` e mostrar nas Definições: "3 registos não subiram — ver detalhe". Nada se perde e a fila não estrangula.

ALTO 4 Eventos de clique acumulam-se na página

Onde: `homeView.js:321`, `quickCapture.js:191`, `locationModal.js:129`

Cada vez que uma destas telas desenha, adiciona um `document.addEventListener('click', ...)` e **nunca o remove**. O ecrã Hoje redesenha a cada gravação. Depois de 20 gravações há 20 escutas de clique acumuladas. A app fica progressivamente mais lenta ao longo do dia — e um técnico não fecha a app.

Correção

Guardar a referência da função e chamar `document.removeEventListener` no fecho. Ou melhor: usar um único ouvinte global de cliques e decidir pelo alvo (delegação de eventos).

ALTO 5 Fotos guardadas duas vezes, em dois formatos

Onde: `src/db/reportsRepo.js:9` e `src/db/db.js:79`

Cada foto é guardada como bytes na base local e convertida para texto base64 no item da fila. O base64 é 33 % maior. Uma foto de 500 KB ocupa 1,17 MB no telemóvel. E não há compressão nenhuma: o `main.js:814` guarda o ficheiro original da câmara — que num telemóvel moderno são 4 a 8 MB.

O documento `PROJECT.md` promete compressão para 1280 px e ~180 KB. Essa compressão não existe no código.

Correção

Comprimir com Canvas antes de guardar (1600 px no lado maior, JPEG a 0,75). Reduz cerca de 95 % do peso, acelera a sincronização e resolve o problema de memória do telemóvel de uma só vez.

ALTO 6 Sem autenticação: qualquer pessoa lê e escreve tudo

Onde: `server.js:60`

Os pontos de acesso `/api/sync/push` e `/api/sync/pull` não pedem credenciais. O CORS está em `*`. Quem souber o endereço apaga ou lê todos os registos de manutenção do estádio.

Também não há registo de *quem* fez o quê: o nome do técnico só existe no `localStorage` do telemóvel dele e nunca chega à base de dados. Para um relatório de manutenção municipal, saber quem interveio não é um extra — é o essencial.

Correção mínima

Um segredo partilhado no cabeçalho do pedido e CORS restrito ao domínio real. Depois, um campo `technician` em cada registo, vindo de um ecrã de entrada simples com PIN.

ALTO 7 Filtro "Pendentes" mostra registos errados

Onde: `src/ui/history.js:91`

A condição `r.status !== 'pending' && r.status !== undefined` deixa passar registos sem estado definido para o filtro "Pendentes". Registos antigos, criados antes da versão 3 do esquema, aparecem onde não devem. Confuso e sem correção óbvia para o técnico.

Correção

Normalizar ao ler: `const status = r.status || 'pending'`. E criar uma migração no Dexie que preenche o estado nos registos antigos, de uma vez.

3. Código Morto ("Ghost Code")

Encontrei três ficheiros e cinco caminhos que ninguém usa. São 1 020 linhas que continuam a ir para o pacote final, a ser lidas por quem trabalha no projeto, e a confundir.

O quê	Linhas	Situação
<code>maintenance_app/card.classList.remove(i.cls))</code>	0	Ficheiro vazio com nome de linha de código. Criado por acidente por um comando mal escapado. Apagar já.
<code>src/ui/stadiumMap.js</code>	321	Nunca importado por ninguém. Substituído pelo <code>stadiumNavigator.js</code> . Apagar.
<code>src/services/audioService.js</code>	160	Nunca importado. A app usa escrita por voz (transcrição), não gravação de áudio. Apagar.
<code>src/ui/dashboard.js</code> + rota <code>metrics</code>	175	A rota <code>metrics</code> existe no <code>main.js:178</code> mas nenhum botão da app navega para lá. Inalcançável. Ou pôr no menu Mais, ou apagar.
Funções vazias de resposta	—	<code>main.js:215 onNewTaskForLocation: () => {}</code> e <code>main.js:226 onConvertToTask: () => {}</code> . O botão "Escolher local do estádio" nas tarefas aparece e não faz nada — o pior tipo de botão.
<code>main.js:190 onNewTask</code>	—	Comentado como "fallback simples por agora". Navega para as tarefas em vez de abrir uma nova. Promessa não cumprida ao utilizador.
<code>PROJECT.md</code> inteiro	~190	Descreve uma estrutura de ficheiros que já não existe (<code>reportForm.js</code> , <code>photoService.js</code> , <code>cloud/firebaseProvider.js</code> , <code>syncQueue.js</code> ...). Documentação que engana é pior que nenhuma. Reescrever ou apagar.

Ganho imediato

Apagar isto são cerca de 1 020 linhas e 20 minutos de trabalho. Zero risco: nada importa estes ficheiros. O pacote final fica mais leve e quem lê o código deixa de perder tempo com caminhos falsos.

4. Código Espagete e Duplicação

4.1 `main.js` é um ficheiro que faz tudo (1 176 linhas)

A classe `App` é ao mesmo tempo: router de navegação, gestor do ciclo de vida, formulário completo de intervenção (185 linhas de HTML em texto), ecrã de definições (200 linhas), janela de salas, e utilitário de escape de HTML. Um ficheiro assim não se pode dividir entre duas pessoas sem conflitos constantes no Git.

Correção

Extraír três ficheiros: `src/ui/reportForm.js` (o formulário completo), `src/ui/settingsView.js` (as definições) e `src/router.js` (a navegação). O `main.js` fica com cerca de 150 linhas: arrancar e ligar as peças. É a mudança com maior retorno de todo o projeto.

4.2 A mesma função `esc()` escrita 16 vezes

Dezasseis ficheiros têm a sua própria cópia de escape de HTML — e em **três variantes diferentes**:

- `main.js:1163` usa `div.textContent` (cria um elemento DOM a cada chamada — lento em listas)
- `homeView.js:18` usa expressão regular com 5 caracteres, incluindo a apóstrofe
- `history.js:323` usa expressão regular com apenas 4 caracteres — **não escapa a apóstrofe**

A terceira variante é um risco de injeção de HTML em atributos delimitados por apóstrofe. Não vejo um ponto explorável hoje, mas o risco existe e é gratuito.

Correção

Um só ficheiro `src/utls/html.js` com `esc()` e `attr()`. Apagar as 16 cópias. Uma implementação, uma correção, um teste.

4.3 Seleção de prioridade copiada quatro vezes

O mesmo bloco de três botões (Baixa / Média / Crítica) com a mesma lógica de cores existe em: `main.js:732`, `quickCapture.js:114`, `tasksView.js:395` e `tasksView.js:568`. Todos escrevem cores diretamente no elemento (`btn.style.borderColor = ...`) em vez de usar uma classe CSS.

Consequência real: as cores escritas assim **não respeitam o tema escuro**. No modo escuro, os botões de prioridade ficam com as cores do tema claro.

Correção

Um componente `PrioritySelector` que só troca classes CSS (`.prio--crit`, `.prio--med`, `.prio--low`). As cores ficam no CSS, onde o tema escuro as consegue redefinir. Remove cerca de 120 linhas repetidas.

4.4 Duas formas de ligar eventos, misturadas

Há 141 usos de `addEventListener` e 16 de `elemento.onclick =`. A segunda forma substitui silenciosamente qualquer ouvinte anterior. Em `main.js:701`, o `bindReportFormEvents()` é chamado uma vez no arranque mas o HTML do formulário é recriado — se alguma vez o formulário for recriado, os eventos desaparecem sem aviso.

4.5 HTML gerado por texto, em todo o lado

Todas as telas constroem interface juntando texto e chamando `innerHTML`. Isto significa: nenhuma verificação em tempo de compilação, escape manual (com as três variantes acima), e redesenho total mesmo quando muda uma linha. O ecrã de definições redesenha as 1 000+ salas do estádio a cada material adicionado (`main.js:481`).

Não recomendo mudar de tecnologia — JavaScript puro é a escolha certa para uma app offline pequena. Mas recomendo dois hábitos: função `html()` com escape automático, e redesenhar só a parte que mudou.

5. Desenho (UI) — Boas Práticas

5.1 Os tokens são muito bons — e são contornados

O `theme.css` é **trabalho de qualidade**: comentários que explicam o porquê, escala tipográfica com mínimo real de 18 px, três blocos de tema na ordem correta, contrastes calculados para 4,5:1. É melhor que a maioria dos sistemas de desenho que vejo.

E depois há **220 estilos escritos à mão dentro do JavaScript** que ignoram tudo isso:

Ficheiro	Estilos inline	Nota
<code>src/ui/homeView.js</code>	62	O ecrã principal é quase todo inline
<code>src/main.js</code>	55	Formulário e definições
<code>src/ui/tasksView.js</code>	27	Tem CSS próprio e ainda usa inline
<code>src/ui/quickCapture.js</code>	27	
<code>src/ui/locationModal.js</code>	24	
<i>Em contraste:</i> <code>toolsView.js</code> , <code>equipmentView.js</code> , <code>notesView.js</code> , <code>moreView.js</code> , <code>bottomNav.js</code> e <code>header.js</code> têm zero — provam que é possível.		

5.2 A escala de letra é ignorada 27 vezes

O tema define `--fs-body: 19px` e proíbe explicitamente valores abaixo de 18 px. Mas o `homeView.js` escreve `font-size: 1.15rem` em 24 sítios — que dá 18,4 px, por acaso aceitável, mas por *coincidência*, não por desenho. E há também `0,7rem` (11,2 px) e `0,75rem` (12 px) no `main.js`: **ilegível de luvas e ao sol**, que é exatamente a situação de uso desta app.

Correção

Substituir todos os `font-size` inline por classes que usam `var(--fs-...)`. Um teste automático que falha se encontrar `font-size` num ficheiro `.js` impede a recaída.

5.3 78 `!important` no `components.css`

Cada `!important` é uma batalha de especificidade perdida — normalmente contra os estilos inline do JS. É um sintoma, não a doença: tirar os inline faz a maioria destes desaparecer sozinha.

5.4 O ecrã Hoje tem duas ações primárias a competir

Em `homeView.js:83-90` há um botão verde grande "Gravar Intervenção Rápida" e logo abaixo outro botão grande "+ Adicionar Fotos ou Materiais (Formulário Completo)". Dois botões grandes lado a lado significam que o técnico tem de *decidir* antes de agir. E o segundo rótulo tem 46 caracteres — ninguém lê isso num telemóvel ao sol.

Correção

Um só botão primário. As fotos entram *depois* de gravar, no detalhe da intervenção. Gravar primeiro, enriquecer depois — é assim que o trabalho de campo funciona.

5.5 Detalhes menores mas visíveis

- **"Bom dia!" a qualquer hora** (`homeView.js:58`) — texto fixo. Às 18h fica errado. Ou usar a hora, ou tirar.
- **Ecrã de arranque de 3 segundos** — `index.html:45` tem uma rede de segurança a 3 s e o CSS anima a saída a 1,6 s. Para uma app que se abre 30 vezes por dia, é meio minuto perdido por dia. Bastam 600 ms.
- **Zoom desativado** — `index.html:5` tem `maximum-scale=1.0`, `user-scalable=no`. Impede um técnico de ampliar para ler uma etiqueta numa foto. Violação direta da norma WCAG 1.4.4.
- **Largura máxima de 480 px** — `main.css:55`. Num tablet de 10", que é o ecrã natural para o escritório de manutenção, fica uma tira estreita no meio do ecrã com cinzento dos dois lados.

6. Utilização e Número de Toques

Contei os toques dos caminhos reais, a partir da app aberta no ecrã Hoje.

Tarefa	Toques	Ideal	Onde se perde
Gravar avaria simples (voz + local por escrever)	3	3	Bom. É o caminho mais importante e está bem.
Gravar avaria com local escolhido da lista	5	4	A lista de locais não mostra os mais usados primeiro. Com 1 000+ salas, é sempre escrever para procurar.
Gravar avaria com foto	8	4	Obriga a passar para o formulário completo, que pede data, hora e minutos gastos antes de deixar tirar a foto.
Marcar tarefa como feita	1	1	Excelente. O alvo de toque tem 48 px e responde logo.
Ver avarias de uma sala específica	6	3	Mais → Áreas do Estádio → setor → sala → ver avarias → voltar. Nenhum atalho a partir do detalhe da avaria.
Gerar relatório mensal em PDF	5	3	Mais → Relatórios → escolher período → gerar → escolher "Guardar como PDF" na janela do sistema.
Adicionar ferramenta ao stock	5	3	Enterrada dois níveis abaixo do menu Mais.

6.1 O maior problema: fotos custam 8 toques

Numa app de manutenção, a foto **é** o registo. Uma foto do disjuntor queimado vale mais que três parágrafos escritos com luvas. Oito toques para chegar lá significa que, na prática, não se tiram fotos.

Correção com maior impacto de toda a revisão

Pôr um botão de câmara

na folha rápida

, ao lado do microfone. Tirar foto → gravar. Total: 4 toques. Data, hora e minutos gastos são opcionais e preenchem-se sozinhos.

6.2 A barra inferior desperdiça um lugar

Quatro abas: Hoje, Intervenções, Tarefas, Mais. O menu "Mais" esconde seis coisas, incluindo Relatórios e Ferramentas — que são de uso diário. Uma aba dedicada a "Registar" (o botão de ação) libertava o ecrã Hoje para ser só informação.

6.3 Sem confirmação visual de que subiu

O indicador no cartão diz "Local" ou "Sincronizado", mas **o campo `synced` nunca é atualizado depois do envio**. O `syncEngine` apaga o item da fila mas nunca chama `reportsRepo.markSynced()`. Todos os cartões dizem "Local" para sempre, mesmo depois de subirem. O técnico não tem forma de saber se o trabalho dele chegou ao servidor.

Correção

Depois de o servidor confirmar, marcar os registos como sincronizados. Uma linha no `syncEngine.js:236` .
Sem isto, o indicador mente — e um indicador que mente é pior que nenhum.

7. Acessibilidade

Ponto	Estado	Nota
Alvos de toque ≥ 48 px	Bom	Consistente. Pensado para luvas.
Contraste de cor	Bom	Tokens calculados para 4,5:1. Bem documentado.
Tamanho mínimo de letra	Parcial	18 px definido, mas furado 27 vezes por inline (até 11 px).
Foco visível	Bom	<code>:focus-visible</code> com contorno de 3 px.
Prender o foco nos modais	Ausente	Nenhum dos 8 modais. Com teclado, o foco escapa para trás do modal.
Fechar com tecla Escape	Ausente	Nenhum modal responde ao Escape.
Zoom do utilizador	Bloqueado	<code>user-scalable=no</code> . Falha WCAG 1.4.4.
<code>aria-live</code> nas mensagens	Parcial	O contentor tem, mas as mensagens não anunciam a gravidade.
Semântica dos modais	Parcial	Só 2 de 8 têm <code>role="dialog"</code> . Nenhum tem <code>aria-modal</code> .
<code>confirm()</code> do sistema	8 usos	Não se pode desenhar nem traduzir. Em modo de app instalada, aparece com o endereço do site — parece um aviso do navegador.

Correção agrupada

Um componente `Modal` partilhado que trata de tudo isto uma só vez: prender foco, Escape, `role="dialog"`, `aria-modal`, devolver o foco ao fechar. Substitui 8 implementações à mão e resolve seis linhas desta tabela de uma vez.

8. Otimização

8.1 Duas consultas que vão piorar com o tempo

- `reportsRepo.getAll()` lê **todos** os registos para a memória e filtra em JavaScript. Com 5 000 avarias e fotos guardadas dentro de cada uma, isso são centenas de megabytes lidos para mostrar cinco cartões no ecrã Hoje.
- `locationsRepo.getAll()` é chamado **a cada abertura** da folha rápida (`quickCapture.js:22`). Mil salas, ordenadas com `localeCompare`, a cada abertura. Deviam ser lidas uma vez e ficar em memória.

Correção

Guardar as fotos numa tabela separada, ligadas por `reportId`. As listas passam a ler registos leves (poucos KB) e só o detalhe carrega as fotos. É a única mudança de estrutura que recomendo — e a que decide se a app aguenta três anos de uso.

8.2 O `components.css` tem 3 079 linhas

Vai inteiro para o telemóvel em cada arranque. Uma boa parte pertence a telas que o técnico raramente abre. Vale a pena separar o essencial (casca, ecrã Hoje, folha rápida) do resto.

8.3 Sincronização a cada 30 segundos, sempre

`syncEngine.js:19` — mesmo com a fila vazia e a app em segundo plano. Gasta bateria por nada. Deve parar quando a app não está visível e acordar ao voltar.

8.4 O ecrã de definições redesenha 1 000 salas por nada

`main.js:481` — adicionar um material chama `renderSettings()`, que recarrega e redesenha toda a base de salas. Deve atualizar só a lista de materiais.

9. O Que Falta — Contexto de Manutenção de um Estádio

Aqui saio da revisão de código e falo do trabalho real. Um estádio municipal tem três realidades que a app ainda não reconhece: **o calendário de jogos manda em tudo, a manutenção preventiva vale mais que a corretiva, e o município pede contas.**

9.1 Etiquetas QR nos equipamentos

É a falta mais importante da app. Uma etiqueta impressa em cada equipamento — quadro elétrico, bomba, projetor, torniquete. O técnico lê com a câmara e a app abre já naquele equipamento, com o histórico dele e o botão de registar avaria.

Isto resolve o problema dos 8 toques e o problema de escolher entre 1 000 salas, ao mesmo tempo. A tabela `equipment` já tem uma coluna `qr_code` preparada no `server/db.js:136` — a base está feita, falta usar. Precisa de: um leitor com a câmara (`BarcodeDetector` , nativo no Android; biblioteca leve no iOS) e um ecrã que imprima folhas de etiquetas.

9.2 Calendário de jogos e janelas de trabalho

Num estádio, *quando* se pode trabalhar é metade do problema. Não se corta a energia da bancada na véspera de um jogo. A app não sabe que há jogo no sábado.

Uma tabela simples de eventos (data, tipo, lotação prevista) e a app passa a poder: marcar tarefas como "antes do jogo" ou "depois do jogo", avisar quando uma tarefa cai num dia de evento, e gerar a lista de verificação obrigatória do dia de jogo — relvado, iluminação, torniquetes, sanitários, saídas de emergência.

9.3 Manutenção preventiva a sério

A app tem tarefas com repetição diária, semanal e mensal — é um bom início. Mas o equipamento de um estádio pede planos com base em **contadores**, não só em datas: horas de funcionamento do gerador, ciclos da bomba de rega, horas de queima dos projetores.

Falta também o que a lei exige e o município audita: inspeção anual do gerador, revisão dos elevadores, teste dos extintores, verificação do sistema de rega antes da primavera. Isso não pode viver na cabeça de uma pessoa.

9.4 Histórico por equipamento e custo

Hoje as avarias ligam-se a *locais*. Deviam ligar-se a *equipamentos*. A pergunta que decide orçamentos é: "*esta bomba já foi arranjada quatro vezes este ano — vale a pena substituir?*". A app ainda não consegue responder.

Com um campo de custo estimado por intervenção (peças + horas), o relatório mensal deixa de ser uma lista de trabalhos e passa a ser um argumento de orçamento para a câmara municipal.

9.5 Registo de quem fez

Já referido como problema de segurança, mas é sobretudo uma falta funcional. Um relatório de manutenção municipal sem o nome do técnico tem valor legal reduzido. E não se consegue distribuir trabalho: não há a quem *atribuir* uma tarefa.

9.6 Stock com aviso de mínimos

A tabela `tools` já tem `min_qty`. Ninguém a usa para avisar. Deveria aparecer no ecrã Hoje: "3 artigos abaixo do mínimo". E ao registar uma intervenção, o material usado devia descontar do stock automaticamente — a tabela `tool_moves` está preparada e vazia.

9.7 Relatório em PDF: falta o essencial

O relatório atual usa a janela de impressão do navegador (`window.print()`), o que funciona mas é frágil: bloqueadores de janelas travam-no, e em modo de app instalada o comportamento é imprevisível. Falta-lhe: assinatura do técnico e do responsável, número de referência sequencial, e o brasão do município ao lado do logótipo da mmcrepo. Um relatório que vai para a câmara precisa de parecer oficial.

9.8 Fotos com data, hora e local gravados na imagem

Numa disputa sobre quando um dano apareceu, a foto vale pelo que prova. Sobrepor data, hora e nome do local no canto da imagem, no momento em que é tirada, torna-a autossuficiente — mesmo depois de sair da app por WhatsApp ou email.

9.9 Outras ideias, por ordem de valor

- **Modo de dia de jogo** — um ecrã simplificado com só a lista de verificação do evento e um botão de avaria urgente.
- **Ligar tarefas a avarias** — o campo `reportId` já existe nas tarefas mas nada o preenche. "Isto precisa de peça" devia criar uma tarefa a partir da avaria.
- **Contratos externos** — elevadores, alarme de incêndio e climatização são normalmente de empresas externas. Saber quem chamar e qual o contrato ativo poupa telefonemas.
- **Leituras de contadores** — água, eletricidade e gás lidos semanalmente detetam fugas cedo. Uma fuga na rega do relvado custa milhares antes de alguém a ver.
- **Exportar para folha de cálculo** — quem faz a contabilidade na câmara quer CSV, não PDF.

10. Plano de Trabalho Sugerido

Ordenado por **valor a dividir pelo esforço**. As fases 1 e 2 são as que eu faria primeiro, sem hesitar.

Fase 1 — Parar as perdas (1 a 2 dias)

Nº	O quê	Esforço	Porquê
1	Corrigir <code>locationId</code> nulo na folha rápida	30 min	Evita perder trabalho
2	Marcar registos como sincronizados após envio	1 h	O indicador deixa de mentir
3	Apagar os 3 ficheiros mortos e o ficheiro de nome absurdo	20 min	–1 020 linhas, risco zero
4	Remover a captura rápida duplicada do ecrã Hoje	2 h	Acaba com os IDs duplicados
5	Limpar as escutas de clique na página	1 h	App deixa de ficar lenta ao longo do dia
6	Reativar o zoom do utilizador	5 min	Acessibilidade e legibilidade
7	Ecrã de arranque de 3 s → 600 ms	10 min	30 abrituras/dia × 2,4 s poupados

Fase 2 — Menos toques onde importa (3 a 4 dias)

Nº	O quê	Esforço	Porquê
8	Botão de câmara na folha rápida (8 → 4 toques)	1 dia	Maior ganho de utilização da app
9	Comprimir fotos com Canvas antes de guardar	4 h	–95 % de peso; cumpre o prometido
10	Locais recentes no topo da lista	3 h	Menos escrita entre 1 000 salas
11	Componente <code>Modal</code> partilhado (foco, Escape, semântica)	1 dia	Resolve 6 falhas de acessibilidade
12	Substituir os 8 <code>confirm()</code> por folha desenhada	3 h	Coerência visual e idioma

Fase 3 — Arrumar a casa (4 a 5 dias)

Nº	O quê	Esforço	Porquê
13	Dividir <code>main.js</code> em formulário + definições + navegação	1,5 dias	Deixa de ser um ficheiro que faz tudo
14	Um só <code>esc()</code> em <code>src/utlis/html.js</code>	3 h	Remove 16 cópias e um risco
15	Componente <code>PrioritySelector</code> partilhado	4 h	Remove 4 cópias; corrige tema escuro

16	Estilos inline → classes CSS (começar pelo <code>homeView</code>)	1,5 dias	Faz o tema escuro funcionar de verdade
17	Contagem de tentativas e fila de itens presos	1 dia	Fila deixa de poder crescer sem fim
18	Reescrever ou apagar o <code>PROJECT.md</code>	2 h	Documentação que engana é pior que nenhuma

Fase 4 — Aguentar o crescimento (3 a 4 dias)

Nº	O quê	Esforço	Porquê
19	Fotos em tabela separada, ligadas por <code>reportId</code>	1,5 dias	Decide se a app aguenta 3 anos
20	Segredo partilhado na API e CORS restrito	4 h	Fecha a porta aberta
21	Campo de técnico em cada registo	4 h	Valor legal do relatório
22	Parar a sincronização com a app em segundo plano	2 h	Bateria

Fase 5 — Aquilo que faz a diferença no terreno (2 a 3 semanas)

Nº	O quê	Esforço	Porquê
23	Etiquetas QR nos equipamentos + leitor	1 semana	A funcionalidade que mais muda o dia a dia
24	Calendário de jogos e janelas de trabalho	4 dias	Alinha a app com a realidade do estádio
25	Histórico e custo por equipamento	3 dias	Transforma relatórios em argumentos de orçamento
26	Avisos de stock mínimo e desconto automático	2 dias	Tabelas já existem e estão vazias
27	Data, hora e local gravados na foto	1 dia	Valor de prova
28	Relatório com assinatura, referência e brasão	2 dias	Aspeto oficial para a câmara

11. O Que Está Bem — e Deve Ficar Como Está

Uma revisão que só aponta defeitos é uma revisão incompleta. Isto está bem feito e não se toca:

- **O `syncEngine.js` tem a filosofia certa.** "Não haver rede é o caso normal, não é uma exceção." A fila nunca se apaga sem confirmação item a item. O crescimento do tempo de espera entre tentativas está bem feito. É o melhor ficheiro do projeto.
- **Datas em hora local, corretamente.** O `tasksRepo.js` faz aritmética de calendário com componentes locais, não em UTC. Uma tarefa criada às 23:30 continua a ser "hoje". Muita gente erra isto.
- **Transações atômicas em todo o lado.** Cada escrita e o respetivo item de fila acontecem na mesma transação Dexie. Não é possível ficar com um registo sem a sua mutação pendente.

- O `theme.css` **está bem construído**. A regra "toda a cor nasce no `:root`" está escrita e cumprida. Contrastes calculados. Comentários que explicam decisões em vez de descreverem código.
- **Os comentários em português explicam o *porquê***. O comentário no `header.js:38` sobre ter retirado a saudação para libertar 120 px é exemplar: diz o que se decidiu, porquê, e o que se ganhou.
- **120 testes unitários a passar**, incluindo testes de esforço com 1 000 registos. A rede de segurança existe.
- **A escrita por voz está bem resolvida**. Voltar a ligar sozinha depois das pausas (`speechService.js:78`) resolve um problema real do Chrome que a maioria das implementações ignora.
- **O Service Worker foi corrigido pela razão certa**. A casca vai à rede primeiro, os ficheiros com nome único vão à cache primeiro. O comentário explica o problema que isso resolveu.

Leitura final

As bases desta app estão bem construídas — dados, sincronização e tokens de desenho estão acima da média. O que falta é *disciplina de camadas*: parar de escrever estilos e HTML dentro da lógica, e deixar de ter duas cópias da mesma coisa. As fases 1 e 2 dão uma app visivelmente melhor em menos de uma semana. A fase 5, e as etiquetas QR em particular, é o que faz o técnico preferir esta app ao caderno.